

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
BỘ MÔN CƠ SỞ DỮ LIỆU PHÂN TÁN



BÁO CÁO BÀI TẬP LỚN
ĐỀ TÀI: HỆ THỐNG BÁN HÀNG TRỰC TUYẾN
ĐA KHO

Giảng viên hướng dẫn	: Kim Ngọc Bách
Họ và tên sinh viên	: Đào Xuân Mai B23DCCN523 Nguyễn Văn Hoàng B23DCCN341 Nguyễn Phạm Hoàng Duy B23DCCN246
Nhóm	: 02

Hà Nội – 2026

Mục lục

Mục lục.....	1
PHẦN I: PHÂN TÍCH NGHIỆP VỤ.....	2
1. Bối cảnh và mục tiêu dự án.....	2
2. Các đối tượng tham gia dự án.....	2
3. Các chức năng chính.....	3
4. Chức năng của các Site.....	4
4.1 Chức năng của Site dữ liệu Kho Bắc/Kho Trung/Kho Nam.....	4
4.2 Chức năng của Site dữ liệu trung tâm.....	4
PHẦN II: THIẾT KẾ LƯỢC ĐỒ.....	5
PHẦN III: PHÂN MẢNH VÀ CẤP PHÁT DỮ LIỆU.....	7
PHẦN IV: CƠ CHẾ NHÂN BẢN VÀ ĐỒNG BỘ DỮ LIỆU.....	11
1. Các bảng thực hiện nhân bản.....	11
2. Lý do.....	11
3. Chiến lược nhân bản.....	11
4. Xử lý Truy vấn đọc tới các bảng nhân bản.....	12
PHẦN V: CÁC TRUY VẤN PHÂN TÁN.....	13
1. Kiểm tra sản phẩm còn hàng ở những kho nào.....	13
2. Tính tổng tồn kho của một sản phẩm trên toàn hệ thống.....	14
3. Thống kê doanh thu theo tháng của từng kho và toàn hệ thống.....	16
4. Tìm top sản phẩm bán chạy nhất toàn hệ thống.....	17
5. Xem đơn hàng có nhiều kiện hàng từ các kho khác nhau.....	19
6. Người dùng xem danh sách sản phẩm trên hệ thống.....	21
PHẦN VI: BÀI TOÁN TRUY XUẤT ĐỒNG THỜI VÀ NHẤT QUÁN TỒN KHO.....	22
1. Tra cứu tồn kho của một sản phẩm trên toàn hệ thống.....	22
2. Đặt hàng khi một kho không đủ số lượng và cần kiểm tra khả năng đáp ứng từ kho khác.....	24
PHẦN VII: ĐÁNH GIÁ HỆ THỐNG.....	26
1. Đánh giá hiệu năng hệ thống.....	26
2. So sánh phương án tập trung với phân tán.....	28
3. Đề xuất cải tiến hệ thống.....	30

PHẦN I: PHÂN TÍCH NGHIỆP VỤ

1. Bối cảnh và mục tiêu dự án

Trong bối cảnh thương mại điện tử ngày càng phát triển, doanh nghiệp cần xây dựng hệ thống quản lý bán hàng trực tuyến có khả năng phục vụ số lượng lớn khách hàng tại nhiều khu vực khác nhau. Doanh nghiệp hiện sở hữu 3 kho hàng chính đặt tại miền Bắc, miền Trung và miền Nam, mỗi kho chịu trách nhiệm quản lý tồn kho và xử lý đơn hàng trong khu vực phụ trách.

Hệ thống cần cho phép khách hàng tra cứu sản phẩm, kiểm tra tồn kho trên toàn hệ thống, đặt hàng và theo dõi trạng thái giao hàng. Đồng thời, doanh nghiệp cần có khả năng quản lý tập trung thông tin sản phẩm, khách hàng và thống kê doanh thu của toàn bộ hệ thống.

Để đáp ứng yêu cầu đó, hệ thống được xây dựng theo mô hình cơ sở dữ liệu phân tán với:

- 01 site dữ liệu trung tâm dùng để quản lý dữ liệu dùng chung và tổng hợp báo cáo.
- 03 site dữ liệu cục bộ tương ứng với Kho Bắc, Kho Trung và Kho Nam dùng để quản lý tồn kho và xử lý nghiệp vụ tại từng khu vực.

Mô hình này giúp:

- Tăng tốc độ truy xuất dữ liệu tại từng khu vực.
- Giảm tải cho hệ thống trung tâm.
- Hỗ trợ xử lý đơn hàng phân tán.
- Đảm bảo khả năng tra cứu dữ liệu thống nhất trên toàn hệ thống.
- Hạn chế tình trạng âm tồn kho khi có nhiều giao dịch đồng thời.

2. Các đối tượng tham gia dự án

a) Khách hàng

Khách hàng là người sử dụng hệ thống để tìm kiếm và đặt mua sản phẩm.

Khách hàng có các chức năng:

- Đăng nhập hệ thống bằng username.
- Xem danh sách sản phẩm và danh mục sản phẩm.
- Tra cứu tồn kho theo từng kho hoặc trên toàn hệ thống.
- Tạo đơn hàng mua sản phẩm.

- Xem danh sách đơn hàng của mình.
- Theo dõi trạng thái đơn hàng.

b) Admin

Admin là người quản trị toàn bộ hệ thống bán hàng trực tuyến đa kho.

Admin có các chức năng:

- Quản lý sản phẩm.
- Quản lý danh mục sản phẩm.
- Quản lý khách hàng.
- Quản lý tồn kho tại các kho.
- Quản lý trạng thái kiện hàng.
- Theo dõi đơn hàng trên toàn hệ thống.
- Xem báo cáo doanh thu theo từng kho, từng vùng và toàn hệ thống.
- Đồng bộ và theo dõi dữ liệu giữa các site.

c) Quản lý kho

Quản lý kho là người phụ trách quản lý hoạt động của kho hàng.

Quản lý kho có các chức năng:

- Xem báo cáo doanh thu theo kho.
- Theo dõi các kiện hàng của kho.
- Quản lý tồn kho tại các kho.

3. Các chức năng chính

Hệ thống có các chức năng chính như sau:

- Quản lý kho hàng.
- Quản lý sản phẩm.
- Quản lý danh mục sản phẩm.
- Quản lý khách hàng.
- Quản lý đơn hàng.
- Xử lý đặt hàng.
- Kiểm tra tồn kho theo từng kho và toàn hệ thống.

- Cập nhật trạng thái đơn hàng.
- Thống kê doanh thu theo kho, theo vùng và toàn hệ thống.

4. Chức năng của các Site

4.1 Chức năng của Site dữ liệu Kho Bắc/Kho Trung/Kho Nam

a) Chức năng quản lý kiện hàng

- Giúp Quản lý kho thực hiện các thao tác như xem, cập nhật (sửa) trạng thái các thông tin của kiện hàng tại khu vực đó. Các thông tin sẽ được lưu trong cơ sở dữ liệu phân mảnh cục bộ.
- Các thông tin quản lý bao gồm:
 - + Mã kiện hàng.
 - + Mã đơn hàng gốc.
 - + Trạng thái kiện (Pending, Shipping, Delivered).
 - + Mã kho phụ trách xuất hàng tương ứng.
 - + Ngày tạo.
 - + Ngày giao.

b) Chức năng quản lý tồn kho

- Giúp Quản lý kho thực hiện các thao tác như xem, thêm, sửa số lượng thông tin tồn kho thực tế. Các thông tin sẽ được lưu trong cơ sở dữ liệu phân mảnh cục bộ.
- Các thông tin tồn kho bao gồm:
 - o Mã sản phẩm
 - o Mã kho
 - o Số lượng tồn kho
 - o Ngày cập nhật.

4.2 Chức năng của Site dữ liệu trung tâm

a) Có toàn bộ chức năng của các site khác

b) Chức năng quản lý thông tin khách hàng

- Giúp Admin thực hiện các thao tác như đọc, thêm, sửa, xóa các thông tin chi tiết của khách hàng. Các thông tin sẽ được lưu trữ trong cơ sở dữ liệu trung tâm.
- Các thông tin của khách hàng bao gồm:
 - + Tên đăng nhập (Username).
 - + Họ và tên.

c) Chức năng quản lý thông tin sản phẩm

- Giúp Admin thực hiện các thao tác như thêm, đọc, sửa, xóa các thông tin chi tiết của sản phẩm. Các thông tin sẽ được lưu trữ trong cơ sở dữ liệu trung tâm. Dữ liệu của sản phẩm thêm vào tại site dữ liệu Trung tâm sẽ được chuyển đến site dữ liệu tương ứng tại Kho Bắc, Kho Trung, Kho Nam.
- Các thông tin của sản phẩm bao gồm:
 - + Mã sản phẩm.
 - + Tên sản phẩm.
 - + Mã danh mục.
 - + Giá bán.

d) Chức năng quản lý giao dịch Đơn hàng

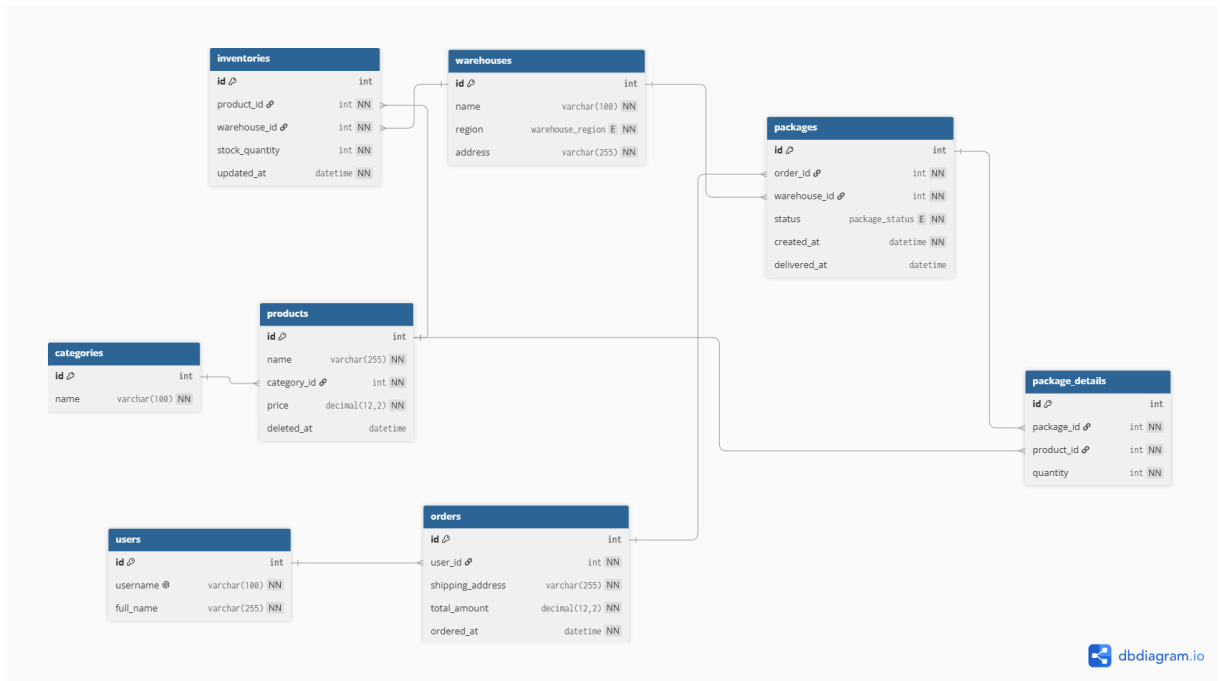
- Giúp hệ thống tự động tiếp nhận thao tác đặt mua sản phẩm của khách, xử lý phân tán để kiểm tra tồn kho và chia tách thành các kiện hàng.
- Các thông tin của đơn hàng bao gồm:
 - + Mã đơn hàng.
 - + Mã khách hàng.
 - + Địa chỉ giao hàng.
 - + Tổng tiền.
 - + Ngày tạo.

e) Quản lý chi tiết kiện hàng

- Giúp lưu thông tin các sản phẩm thuộc từng kiện hàng.
- Các thông tin gồm:
 - + Mã kiện hàng.
 - + Mã sản phẩm.
 - + Số lượng.

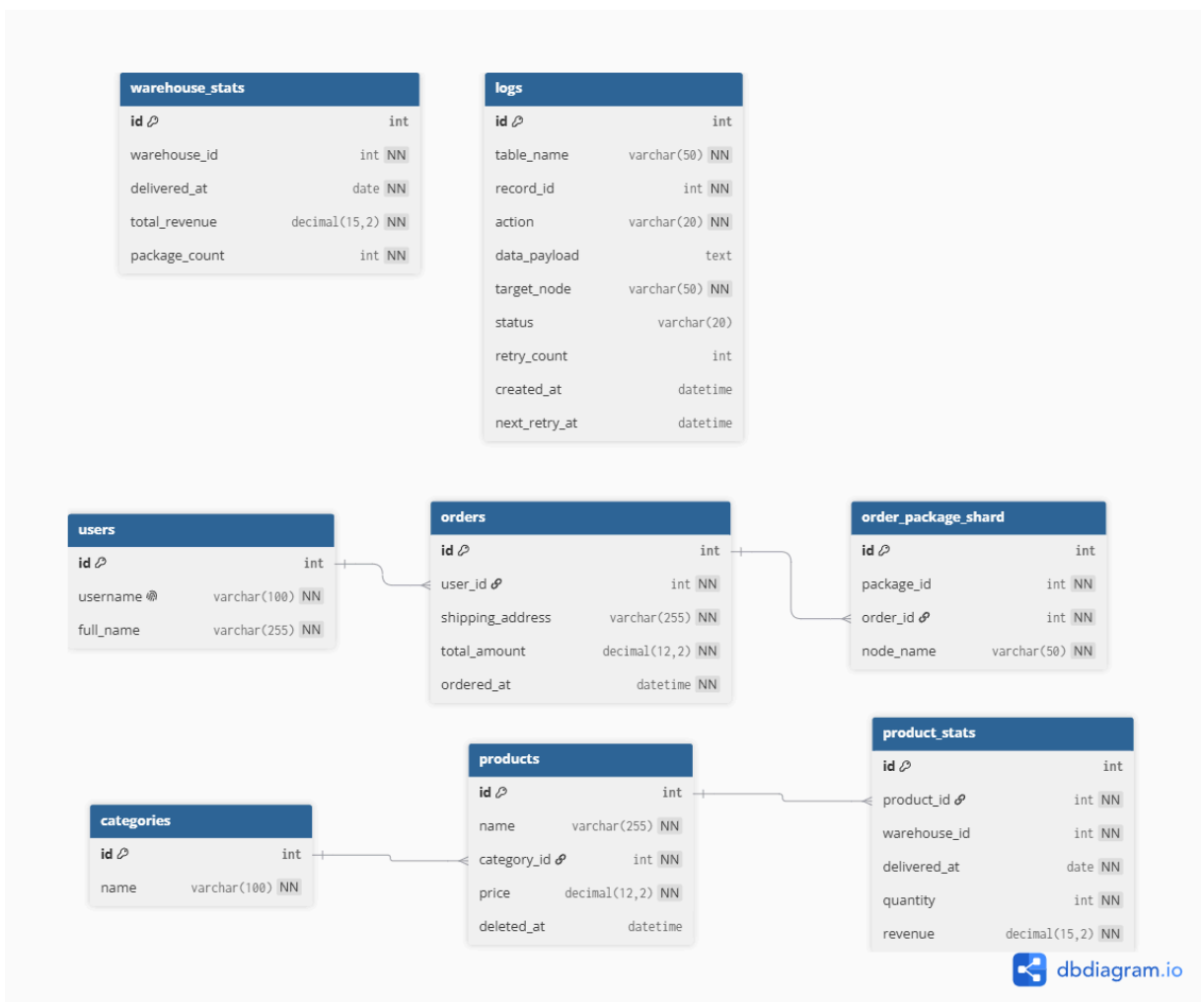
PHẦN II: THIẾT KẾ LƯỢC ĐỒ

1.Lược đồ toàn cục



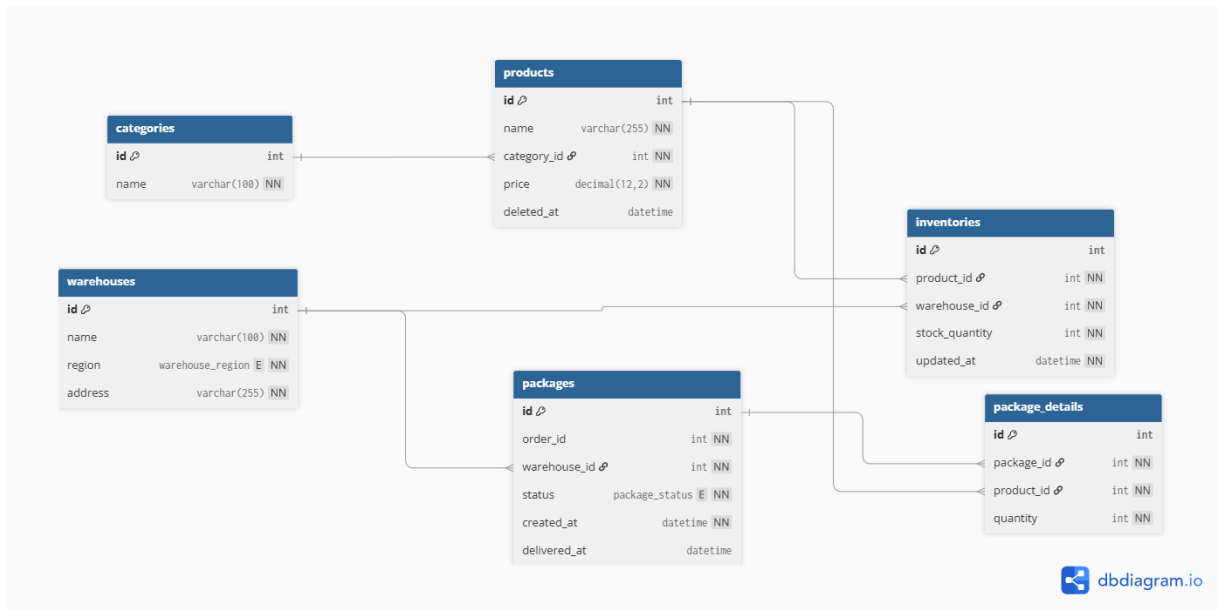
Link : [Lược đồ toàn cục.png](#)

2. Lược đồ site dữ liệu trung tâm



Link : [Lược đồ site dữ liệu trung tâm.png](#)

3.Lược đồ site dữ liệu cục bộ



Link: [Lược đồ site dữ liệu cục bộ.png](#)

PHẦN III: PHÂN MẢNH VÀ CẤP PHÁT DỮ LIỆU

1. Phân tích yêu cầu và Đề xuất phương án phân mảnh dữ liệu

1.1. Phân tích yêu cầu nghiệp vụ

Dựa trên đặc thù hoạt động bán hàng đa kho trực tuyến, dữ liệu hệ thống được phân tích dựa trên hai nhóm yêu cầu vận hành chính:

- Hoạt động cục bộ tại từng miền (Bắc, Trung, Nam): Nhà kho (bảng warehouses) được chuyển đổi sang phân mảnh ngang nguyên thủy để phân tán về các node phụ. Hoạt động cập nhật tồn kho (bảng inventories) và xử lý kiện hàng (bảng packages, package_details) diễn ra tại từng kho hàng thuộc các khu vực khác nhau. Để tối ưu tốc độ xử lý cục bộ và giảm thiểu độ trễ kết nối, các bảng dữ liệu này cần được phân tán ngay tại site khu vực bằng cơ chế Phân mảnh ngang.
- Quản lý toàn cục tại trung tâm: Thông tin tài khoản (bảng users), dữ liệu sản phẩm (bảng products, categories), và thông tin đơn hàng tổng quát (bảng

orders) kèm bảng ánh xạ định tuyến (bảng `order_package_shards`) cần được quản lý thống nhất để đảm bảo tính nhất quán và phục vụ đối soát doanh thu toàn hệ thống. Do đó, các bảng này được cấu hình lưu trữ tập trung tại Site Trung tâm.

1.2. Đề xuất phương án phân mảnh dữ liệu cụ thể

Hệ thống giữ nguyên cấu trúc cột của các bảng tại tất cả các site (không sử dụng phân mảnh dọc). Phương án phân mảnh và lưu trữ cụ thể như sau:

a, Các bảng lưu trữ tập trung:

- Danh sách bảng: `users`, `orders`, `order_package_shards`, `products`, `categories`, `product_stats`, `warehouse_stats`, `logs`.
- Vị trí lưu trữ: Site Trung tâm (`main_db`).
- Lý do: Đảm bảo tính nhất quán toàn cục cho các dữ liệu ít biến động hoặc cần đối soát tập trung. Bảng `order_package_shards` đóng vai trò Shard Map (bảng ánh xạ vị trí lưu trữ thực tế của kiện hàng) làm cơ sở cho bộ định tuyến truy vấn của hệ thống. Bảng `product_stats` và `warehouse_stats` đóng vai trò là kho dữ liệu thống kê, lưu trữ các chỉ số về hiệu suất bán hàng được tổng hợp định kỳ từ các cơ sở dữ liệu site miền. Cuối cùng, bảng `logs` đóng vai trò là hàng đợi thông điệp Outbox (Outbox Queue), ghi nhận toàn bộ lịch sử thay đổi dữ liệu dùng chung để làm cơ sở cho tiến trình Replication Worker đồng bộ, cập nhật xuống các site miền.

b, Bảng phân mảnh ngang nguyên thủy:

- Danh sách bảng: `warehouses`.
- Vị trí lưu trữ: Phân mảnh ngang và lưu tại 3 Site miền (Bắc lưu tại `north`, Trung tại `central`, Nam tại `south`) dựa trên thuộc tính quyết định phân mảnh `region`. Không lưu trữ bảng vật lý `warehouses` tại Main DB.
- Lý do: Chuyển đổi từ mô hình tập trung sang phân tán nguyên thủy. Các chi nhánh tự quản lý thông tin nhà kho của mình cục bộ và thiết lập các khóa ngoại vật lý cứng trực tiếp (như `inventories.warehouse_id -> warehouses.id`, `packages.warehouse_id -> warehouses.id`) tại từng node chi nhánh để tối ưu hóa hiệu năng, giảm tải cho Main DB và đảm bảo tính tự trị dữ liệu của từng site.

c, Các bảng phân mảnh ngang dẫn xuất:

- Bảng inventories (Tồn kho sản phẩm): Được phân mảnh dựa trên khóa ngoại warehouse_id liên kết với bảng warehouses. Tồn kho của các kho thuộc miền nào sẽ được lưu tại site miền đó (miền Bắc lưu tại north, miền Trung tại central, miền Nam tại south). Điều này giúp các miền tự chủ trong giao dịch cập nhật tồn kho thời gian thực, loại bỏ tranh chấp khóa diện rộng.
- Bảng packages (Kiện hàng) và package_details (Chi tiết kiện hàng): Được phân mảnh dựa trên khóa ngoại warehouse_id liên kết với bảng warehouses. Kiện hàng và chi tiết sản phẩm trong kiện xuất phát từ nhà kho thuộc miền nào sẽ được lưu trữ cục bộ tại site miền của kho hàng đó. Điều này cho phép nhân viên tại các kho thuộc cùng một miền thực hiện đóng gói và cập nhật trạng thái giao hàng (Pending -> Shipping -> Delivered) cục bộ độc lập mà không cần kết nối WAN về trung tâm.

2. Giải thích lựa chọn phân mảnh và cơ chế xử lý trong mã nguồn

Thiết kế phân mảnh dữ liệu của hệ thống được hiện thực hóa và tối ưu trực tiếp trong mã nguồn Backend (FastAPI) qua các cơ chế sau:

2.1. Phân tách nghiệp vụ Đơn hàng và Kiện hàng

- Nghiệp vụ: Đơn hàng (orders) đại diện cho giao dịch toàn cục nên được lưu tập trung tại Site Trung tâm. Tuy nhiên, một đơn hàng có thể chứa các sản phẩm từ các kho ở nhiều miền khác nhau.
- Giải pháp: Mã nguồn (OrderService.create_order) thực hiện phân bổ tồn kho tối ưu và chia tách đơn hàng thành nhiều kiện hàng (packages) tương ứng với các miền chịu trách nhiệm xuất. Dữ liệu packages và package_details được ghi trực tiếp vào các site miền tương ứng (north, central, south) nhằm phân tán tải ghi và hỗ trợ xử lý đóng gói độc lập tại mỗi kho hàng. Đồng thời, bảng trung gian order_package_shards (Shard Map) được lưu tại trung tâm để định vị kiện hàng.

2.2. Định tuyến truy vấn

Mã nguồn điều phối kết nối database linh hoạt để tối ưu hiệu năng:

- Định tuyến trực tiếp: Khi biết rõ vị trí dữ liệu (ví dụ đọc tồn kho của một kho hàng cụ thể hoặc đọc chi tiết kiện hàng của đơn hàng dựa trên Shard Map), Backend chỉ mở kết nối trực tiếp đến site miền chứa dữ liệu đó, không truy vấn dư thừa sang các site khác.
- Phân tán thu thập: Khi cần lấy dữ liệu tổng hợp (như tổng tồn kho của sản phẩm trên toàn quốc), Backend sử dụng ThreadPoolExecutor để gửi đồng thời các câu lệnh truy vấn song song đến cả 3 database miền, sau đó tổng hợp kết quả để giảm độ trễ phản hồi xuống mức tối thiểu.

2.3. Đảm bảo tính nhất quán giao dịch

- Thách thức: Tránh tình trạng đặt hàng đồng thời gây âm tồn kho và lỗi không đồng bộ dữ liệu giữa Site Trung tâm và các Site miền.
- Giải pháp: Trong quy trình tạo đơn hàng, cập nhật tồn, Backend sử dụng cơ chế Khóa bi quan (Pessimistic Locking) bằng câu lệnh SELECT ... FOR UPDATE trên bảng inventories ở các database miền để khóa các bản ghi tồn kho liên quan trong suốt quá trình xử lý giao dịch. Giao dịch liên kết này được commit đồng loạt trên tất cả các database (Main DB và các DB miền). Nếu bất kỳ node nào gặp lỗi hoặc không đủ hàng, Backend lập tức phát lệnh rollback toàn bộ giao dịch để đảm bảo tính toàn vẹn dữ liệu.

2.4. Khả năng chịu lỗi và tính sẵn sàng cao

- Vấn đề: Tránh việc hệ thống bị treo luồng khi một trong các database miền bị ngắt kết nối hoặc gặp sự cố vật lý.
- Giải pháp: Lóp kết nối database của Backend tích bộ ngắt mạch tự động với thời gian chờ kết nối 0.5 giây. Nếu một site miền không phản hồi, hệ thống sẽ đánh dấu site đó là OFFLINE và lập tức chặn toàn bộ kết nối thử đến site này trong 10 giây tới, trả về kết quả dự phòng nhanh chóng. Sau đó, hệ thống sẽ tự động thử kết nối lại để khôi phục trạng thái hoạt động bình thường khi sự cố được khắc phục.

PHẦN IV: CƠ CHẾ NHÂN BẢN VÀ ĐỒNG BỘ DỮ LIỆU

1. Các bảng thực hiện nhân bản

Trong kiến trúc cơ sở dữ liệu phân tán của hệ thống, hai bảng dữ liệu cốt lõi được lựa chọn để thực hiện nhân bản là:

- Category (Danh mục sản phẩm)
- Product (Sản phẩm)

2. Lý do

- **Thống nhất định danh:** Đảm bảo tất cả các kho trên toàn quốc đều sử dụng chung một mã sản phẩm (ID), tên gọi và quy chuẩn danh mục. Điều này cực kỳ quan trọng khi thực hiện điều chuyển hàng hóa giữa các kho.
- **Tối ưu hóa truy xuất cục bộ:** Hai bảng này có đặc thù là Read-Heavy (Đọc nhiều - Ghi ít). Dữ liệu hiếm khi thay đổi nhưng lại được tra cứu liên tục hàng ngàn lần mỗi ngày bởi nhân viên kho. Việc lưu trữ bản sao dữ liệu ngay tại máy chủ cục bộ giúp tốc độ hiển thị danh sách sản phẩm đạt mức gần như tức thì. Khắc phục hoàn toàn độ trễ và rủi ro gián đoạn quy trình kho bãi khi đường truyền WAN về trụ sở chính bị đứt. Tương tự như khi có vô số truy vấn đọc đến từ khách hàng, việc có nhiều site để thay phiên nhau cũng tránh việc quá tải.
- **Kiểm soát quyền sửa tập trung:** Chỉ Admin tại Site Trung tâm mới có quyền thêm sản phẩm mới hoặc sửa thông tin sản phẩm/danh mục. Site chi nhánh chỉ có quyền "Đọc" để đảm bảo các chi nhánh không tự ý thay đổi thông tin sản phẩm, gây sai lệch báo cáo tài chính toàn hệ thống.
- **Ràng buộc khóa ngoại:** Tại các CSDL chi nhánh, các bảng phân mảnh theo khu vực như Inventory (Tồn kho) hay PackageDetail (Chi tiết kiện hàng) bắt buộc phải tham chiếu đến product_id. Nếu không nhân bản bảng Product xuống chi nhánh, hệ thống sẽ không thể thiết lập khóa ngoại và không thể đảm bảo toàn vẹn dữ liệu cục bộ. Hơn nữa, kích thước của dữ liệu danh mục là khá nhỏ, hoàn toàn tối ưu để nhân bản toàn phần mà không gây tốn dung lượng ổ đĩa.

3. Chiến lược nhân bản

Dựa trên yêu cầu của bài toán, nhóm đề xuất chiến lược Nhân bản Toàn phần kết hợp Bất đồng bộ theo mô hình Master-Slave để triển khai cho 2 bảng trên. Chiến lược này được cấu thành từ 3 yếu tố trọng tâm:

1. Về phạm vi dữ liệu (Full Replication): Áp dụng Nhân bản Toàn phần. Thay vì phân mảnh chia nhỏ sản phẩm theo khu vực, toàn bộ danh mục sản phẩm và sản phẩm từ CSDL Trung tâm sẽ được chép nguyên vẹn xuống tất cả các CSDL chi nhánh.

2. Về luồng dữ liệu (Master-Slave Topology / One-way Push): CSDL Trung tâm đóng vai trò là Master độc quyền ghi dữ liệu. Các CSDL chi nhánh đóng vai trò là

Slave chỉ để đọc. Luồng dữ liệu đi một chiều từ trên xuống dưới. Các Node chi nhánh tuyệt đối không được phép thực hiện lệnh chỉnh sửa lên 2 bảng này để tránh tình trạng xung đột dữ liệu.

3. Về cơ chế đồng bộ (Asynchronous Log-based Worker): Vì đã chặn Thiết kế mô hình Bất đồng bộ thông qua bảng Log (logs) kết hợp cơ chế Event-driven:

- Phân rã Nhật ký: Mỗi khi có thao tác Thêm/Sửa/Xóa tại Main DB, hệ thống sẽ tạo ra N bản ghi nhật ký độc lập (N tương ứng với số lượng node chi nhánh) với trạng thái ban đầu là PENDING.
- Đồng bộ Tức thời: Ngay sau khi Main DB commit(), nó sẽ gửi một tín hiệu đánh thức cho Background Worker. Worker sẽ lập tức đọc các bản log PENDING và đẩy dữ liệu xuống các chi nhánh.
- Cơ chế Chịu lỗi Độc lập: Quá trình đồng bộ xuống các node hoàn toàn độc lập với nhau. Ví dụ nếu node Central bị đứt mạng, truy vấn đồng bộ tới Central sẽ thất bại. Lúc này, chỉ bản log của Node Central bị đánh dấu FAILED và tăng biến đếm retry_count, các Node North và South vẫn được cập nhật bình thường.

Background Worker sẽ tự động thức dậy mỗi 60 giây để quét và thử lại việc đồng bộ cho những bản log bị lỗi. Nếu thử lại quá 3 lần mà Node vẫn sập, Worker sẽ bỏ cuộc, ghi nhận log này là lỗi vĩnh viễn và dừng việc quét lại nó. Đồng thời, kích hoạt tín hiệu WebSocket về màn hình Admin Tổng để báo lại.

- API Đồng bộ Thủ công: Ngoài cơ chế tự động qua Log, nhóm cung cấp thêm một API chuyên dụng để Admin có thể đồng bộ thủ công. API này hoạt động theo cơ chế rà soát toàn diện: Tải toàn bộ dữ liệu từ Main DB lên RAM, so sánh từng dòng (row-by-row) với Node chi nhánh, tự động chèn dòng thiếu và sửa dòng sai. Đưa Node chi nhánh trở lại trạng thái hoàn hảo 100%.

4. Xử lý Truy vấn đọc tới các bảng nhân bản

Để tối ưu hóa luồng dữ liệu mà vẫn đảm bảo tính nhất quán, hệ thống tích hợp thuật toán Định tuyến đọc thông minh kết hợp Circuit Breaker:

(Cơ chế này chỉ áp dụng chọn lọc cho các API tra cứu dữ liệu chung phân tán như: Xem danh mục, Xem toàn bộ sản phẩm và Xem sản phẩm theo danh mục)

- Phân luồng theo Vai trò: Frontend tự động đánh giá vai trò người dùng để đính kèm HTTP Header X-Target-Node. Nếu là Khách hàng phổ thông (user), giá trị sẽ là auto để hệ thống chia sẻ tải. Ngược lại, với tất cả các vai trò quản lý (Admin, Quản lý kho, v.v.), giá trị luôn được gán là main để đọc trực tiếp dữ liệu tổng từ Site Trung tâm.
- Cân bằng tải ngẫu nhiên & Kiểm tra sức khỏe: Khi Backend nhận yêu cầu từ Khách hàng (auto), hệ thống sẽ xáo trộn 3 node chi nhánh và chọn ngẫu nhiên một node theo cơ chế Random. Trước khi trả về kết nối, Circuit Breaker sẽ thực hiện một truy vấn rỗng để "Ping" kiểm tra xem Node đó có đang thực sự ONLINE hay không.

- Bể lái tự động & Cảnh báo thời gian thực: Nếu thao tác "Ping" thất bại, Circuit Breaker lập tức đánh dấu Node đó là OFFLINE, hủy kết nối lỗi, phát cảnh báo tức thời qua WebSocket đến màn hình Admin, và lập tức thử kết nối sang node chi nhánh khác còn lại trong danh sách. Nếu tất cả các node chi nhánh đều sập, hệ thống sẽ tự động "bể lái" truy cập hoàn toàn về Main DB, đảm bảo trải nghiệm người dùng không bao giờ bị gián đoạn.

PHẦN V: CÁC TRUY VẤN PHÂN TÁN

1. Kiểm tra sản phẩm còn hàng ở những kho nào

1.1. Mô tả yêu cầu nghiệp vụ

Khi khách hàng hoặc quản trị viên tra cứu chi tiết một sản phẩm trên hệ thống, ứng dụng cần hiển thị đầy đủ danh sách các kho hàng hiện tại đang còn lưu trữ sản phẩm này, kèm theo số lượng tồn kho khả dụng tương ứng tại mỗi kho.

Chi Tiết Sản Phẩm

Sản phẩm 1

Danh mục: **Danh mục 1**

Giá bán: **100.000 VNĐ**

Tổng tồn kho: **6**

Tồn Kho Tại Các Kho

Kho Hàng	Số Lượng	Cập Nhật Cuối
Kho Nghệ An	2	20:20:20 29/5/2026
Kho Hà Nội	1	21:22:25 29/5/2026
Kho Hải Phòng	1	21:22:09 29/5/2026
Kho TP Hồ Chí Minh	2	20:20:20 29/5/2026

Đóng

1.2. Phân tích dữ liệu phân tán liên quan

- Bảng dữ liệu chính: Bảng inventories (Tồn kho sản phẩm), được phân mảnh ngang dẫn xuất và lưu trữ tại 3 database miền (north_db, central_db, south_db). Mỗi database miền chỉ lưu tồn kho của các nhà kho thuộc miền đó.

- Bảng dữ liệu hỗ trợ: Bảng warehouses (Kho hàng), được phân mảnh ngang nguyên thủy trên các database miền (north_db, central_db, south_db). Để lấy thông tin tên kho, hệ thống thực hiện gom dữ liệu từ tất cả các Node phụ (Scatter-Gather) thông qua WarehouseRepository.

Thách thức phân tán: Dữ liệu tồn kho nằm phân tán ở 3 database miền khác nhau. Hệ thống phải truy vấn cả 3 database này mới thu thập đủ thông tin tồn kho toàn cục của một sản phẩm.

1.3. Cơ chế xử lý truy vấn phân tán trong mã nguồn (Scatter-Gather)

Hệ thống sử dụng mô hình Scatter-Gather (Phân tán Gom lô) thông qua hàm `InventoryService.list_by_product` như sau:

- Bước 1 (Scatter): Backend nhận mã sản phẩm (`product_id`) và sử dụng `ThreadPoolExecutor` để gửi đồng thời các câu lệnh truy vấn song song tới cả 3 database miền (north, central, south).
- Bước 2 (Query): Tại mỗi database miền, Backend thực thi câu lệnh SQL `SELECT * FROM inventories WHERE product_id = product_id` để lấy toàn bộ danh sách tồn kho của sản phẩm tại các kho hàng thuộc miền đó.
- Bước 3 (Gather): Backend thu thập kết quả trả về từ các luồng (chờ tối đa 0.5 giây để tránh treo luồng nếu có site bị sập).
- Bước 4 (Map): Backend ghép kết quả tồn kho của 3 miền và ghép với dữ liệu bảng warehouses thu thập được từ các Node chi nhánh để hiển thị tên kho và số lượng tương ứng cho người dùng.

Khả năng chịu lỗi: Nếu một site miền bị sập, Circuit Breaker sẽ chặn kết nối tới site đó và trả về kết quả từ các miền còn lại.

2. Tính tổng tồn kho của một sản phẩm trên toàn hệ thống

2.1. Mô tả nghiệp vụ

Khi người dùng xem chi tiết sản phẩm, hệ thống hiển thị tổng số lượng tồn kho toàn cục của sản phẩm đó. Thông tin này hỗ trợ khách hàng xác định khả năng cung ứng trước khi đặt đơn hàng, đồng thời giúp nhà quản lý theo dõi tổng lượng hàng hóa lưu kho trên toàn hệ thống.

2.2. Phân tích dữ liệu phân tán liên quan

- Bảng dữ liệu liên quan: Bảng inventories lưu trữ chi tiết số lượng tồn kho của từng sản phẩm tại mỗi kho hàng.
- Phương án phân tán: Bảng inventories được phân mảnh ngang dẫn xuất dựa trên thuộc tính vùng miền của các kho hàng và được lưu trữ trực tiếp tại ba cơ sở dữ liệu chi nhánh gồm north, central, south. Cơ sở dữ liệu trung tâm không lưu trữ bảng này.
- Thách thức phân tán: Dữ liệu tồn kho bị phân mảnh vật lý tại các địa điểm khác nhau. Do đó, hệ thống bắt buộc phải thu thập dữ liệu từ tất cả các phân mảnh đang hoạt động để thực hiện phép tính tổng trên kết quả gộp.

2.3. Cơ chế xử lý truy vấn phân tán trong mã nguồn

Mã nguồn Backend thực hiện yêu cầu này thông qua hàm `get_total_stock_by_product` trong lớp `InventoryService` bằng mô hình phân tán và thu thập song song:

- Bước 1: Khi nhận yêu cầu tính tổng tồn kho cho một mã sản phẩm, hệ thống khởi tạo các luồng xử lý song song để gửi truy vấn đồng thời đến ba cơ sở dữ liệu chi nhánh gồm north, central, south.
- Bước 2: Tại mỗi chi nhánh, hệ thống thực hiện truy vấn các bản ghi tồn kho thuộc mã sản phẩm tương ứng và tính tổng số lượng tồn kho cục bộ của chi nhánh đó.
- Bước 3: Hệ thống thu thập kết quả trả về từ các chi nhánh với thời gian chờ tối đa là 0.5 giây, sau đó cộng dồn các giá trị này để xác định tổng số lượng tồn kho toàn hệ thống.

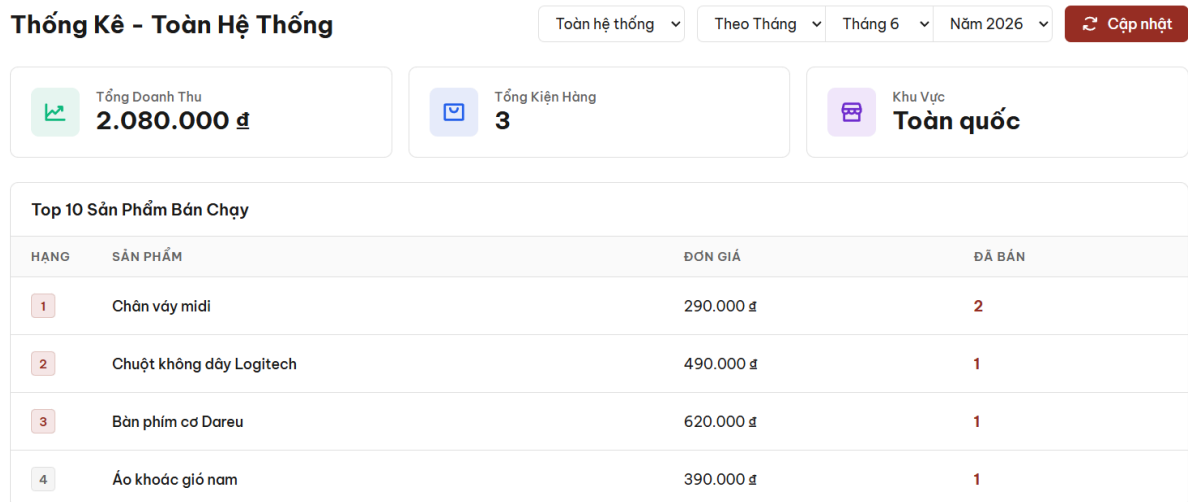
Giải pháp bảo đảm tính sẵn sàng:

- Bộ ngắt mạch: Trước khi gửi truy vấn đến một chi nhánh, hệ thống kiểm tra trạng thái khả dụng qua bộ ngắt mạch. Nếu chi nhánh được xác định đang ngoại tuyến, hệ thống sẽ bỏ qua kết nối đến chi nhánh đó để bảo đảm hiệu năng.
- Xử lý giới hạn thời gian chờ: Nếu chi nhánh gặp sự cố hoặc không phản hồi trong vòng 0.5 giây, hệ thống sẽ tự động ghi nhận số lượng tồn kho tại đó bằng 0 và tiếp tục tổng hợp từ các chi nhánh còn lại nhằm tránh làm gián đoạn toàn bộ yêu cầu.

3. Thống kê doanh thu theo tháng của từng kho và toàn hệ thống

3.1. Mô tả yêu cầu nghiệp vụ

Khi admin truy cập giao diện thống kê của hệ thống, ứng dụng cần hiển thị tổng doanh thu và tổng số lượng kiện hàng đã giao thành công theo khoảng thời gian tùy chọn (ngày, tháng, năm), hỗ trợ lọc chi tiết theo từng kho hàng cụ thể hoặc xem báo cáo tổng hợp toàn hệ thống.



3.2. Phân tích dữ liệu phân tán liên quan

- Bảng dữ liệu gốc (Phân tán): Bảng packages (Kiện hàng) và package_details (Chi tiết kiện hàng) được phân mảnh ngang và lưu trữ tại 3 database miền (north_db, central_db, south_db). Trạng thái giao hàng và thời gian giao hàng thực tế (delivered_at) được ghi nhận trực tiếp tại database miền nơi kho hàng đó trực thuộc.
- Bảng dữ liệu hỗ trợ (Tập trung): Bảng warehouse_stats (Thống kê hiệu suất kho) được lưu trữ tập trung tại Database trung tâm (Main DB). Bảng này lưu trữ thông tin doanh thu và số lượng kiện hàng đã được tổng hợp sẵn theo từng kho hàng và từng ngày giao hàng.

Thách thức phân tán: Dữ liệu đơn hàng và chi tiết sản phẩm bán ra nằm rải rác ở các database miền khác nhau dưới dạng dữ liệu giao dịch (transactional data). Việc tính toán tổng doanh thu theo thời gian thực (real-time) trực tiếp từ các database miền mỗi khi người dùng tải trang thống kê sẽ gây quá tải đường truyền mạng và giảm hiệu năng hệ thống đáng kể, đặc biệt khi có Node phụ bị mất kết nối (offline).

3.3. Cơ chế xử lý và tổng hợp dữ liệu phân tán (Scatter-Gather & Centralization)

Hệ thống sử dụng mô hình kết hợp Scatter-Gather (Phân tán Gom lô) để tổng hợp dữ liệu định kỳ/thủ công và Centralized Querying (Truy vấn tập trung) để hiển thị dữ liệu như sau:

Giai đoạn 1: Tổng hợp dữ liệu phân tán (Scatter-Gather)

Tiến trình này được kích hoạt tự động hàng ngày vào thời gian cố định (cấu hình qua .env) hoặc chạy thủ công khi người dùng bấm nút cập nhật trên giao diện:

- Bước 1 (Scatter): Backend tạo các bản ghi yêu cầu cập nhật (ReplicationLog với thao tác SYNC_STATS) gửi song song tới cả 3 Node phụ miền (north, central, south).
- Bước 2 (Query): Tại mỗi database miền, Backend thực thi truy vấn SQL để gom nhóm và tính toán doanh thu.
- Bước 3 (Gather): Thu thập dữ liệu tính toán từ các luồng của các miền gửi về.
- Bước 4 (Map & Centralize): Thực hiện ghi đè và lưu trữ kết quả tập hợp vào bảng trung tâm warehouse_stats tại Database trung tâm thông qua cú pháp INSERT ... ON DUPLICATE KEY UPDATE để cập nhật doanh thu và số lượng kiện hàng mới nhất của từng ngày.

Giai đoạn 2: Truy vấn dữ liệu thống kê (Centralized Querying)

Khi người dùng tải trang báo cáo doanh thu trên Frontend:

- Backend không cần truy vấn tới các database miền mà thực hiện truy vấn trực tiếp và nhanh chóng trên bảng thống kê tập trung warehouse_stats tại Main DB.
- Trích xuất doanh thu và số lượng kiện hàng theo dải ngày tương ứng với cấu hình thời gian người dùng lọc (theo Ngày, tháng hoặc năm).

Khả năng chịu lỗi và hiệu năng: Giao diện thống kê tải tức thời (tối ưu hóa trải nghiệm UI) và hoàn toàn không bị ảnh hưởng hay treo luồng ngay cả khi các Node chi nhánh đang ngoại tuyến (offline).

4. Tìm top sản phẩm bán chạy nhất toàn hệ thống

4.1. Mô tả yêu cầu nghiệp vụ

Khi admin truy cập trang thống kê, hệ thống cần hiển thị danh sách Top 10 sản phẩm có số lượng bán ra nhiều nhất (xếp hạng giảm dần theo số lượng) trong một khoảng thời gian được chọn (ngày, tháng, năm), hỗ trợ lọc theo kho hàng hoặc hiển thị toàn bộ hệ thống.

4.2. Phân tích dữ liệu phân tán liên quan

- Bảng dữ liệu gốc (Phân tán): Dữ liệu chi tiết về số lượng sản phẩm bán ra nằm trong bảng package_details (Chi tiết kiện hàng) liên kết với packages (Kiện hàng), phân mảnh và lưu trữ độc lập tại các database miền (north_db, central_db, south_db).

Bảng dữ liệu hỗ trợ (Tập trung):

- Bảng product_stats (Thống kê sản phẩm bán ra) được lưu trữ tập trung tại Main DB, lưu trữ tổng số lượng (quantity) và doanh thu (revenue) đã được tổng hợp của từng sản phẩm theo kho hàng và theo ngày giao hàng.
- Bảng products (Thông tin sản phẩm) lưu trữ tập trung thông tin tên sản phẩm, danh mục và đơn giá tại Main DB.

Thách thức phân tán: Để tính toán ra lượng bán chạy của từng sản phẩm trên toàn quốc, hệ thống phải cộng dồn số lượng bán ra của sản phẩm đó từ tất cả các kho thuộc 3 vùng miền khác nhau. Việc truy vấn phân tán thời gian thực (real-time) và tính hàm SUM() trực tiếp trên các bảng giao dịch dung lượng lớn ở các Node phụ sẽ gây nghẽn băng thông và có nguy cơ thất bại cao nếu một trong các Node gặp sự cố.

4.3. Cơ chế xử lý và tổng hợp dữ liệu phân tán (Scatter-Gather & Centralization)

Hệ thống kết hợp cơ chế Scatter-Gather (Phân tán Gom lô) để tổng hợp số liệu về DB trung tâm trước và Centralized Querying (Truy vấn tập trung) kèm liên kết bảng (JOIN) khi người dùng hiển thị:

Giai đoạn 1: Tổng hợp dữ liệu sản phẩm bán ra (Scatter-Gather)

Hoạt động định kỳ hàng ngày hoặc kích hoạt thủ công từ quản trị viên:

- Bước 1 (Scatter): Hệ thống tạo các lệnh yêu cầu cập nhật (ReplicationLog với thao tác SYNC_STATS) gửi song song tới cả 3 Node phụ miền.
- Bước 2 (Query): Tại từng database miền, hệ thống thực thi câu lệnh SQL để tổng hợp số lượng bán ra của từng sản phẩm:
- Bước 3 (Gather): Gom toàn bộ dữ liệu thống kê sản phẩm trả về từ các miền.
- Bước 4 (Map & Centralize): Ghi đè số liệu tổng hợp vào bảng tập trung product_stats trên Main DB thông qua cú pháp INSERT ... ON DUPLICATE KEY UPDATE.

Giai đoạn 2: Trích xuất xếp hạng bán chạy (Centralized Querying & Join)

Khi người dùng tải trang thống kê top sản phẩm:

- Truy vấn thống kê: Backend thực hiện truy vấn duy nhất trên bảng tập trung product_stats ở Main DB để tính tổng số lượng bán ra và sắp xếp giảm dần, lấy ra 10 sản phẩm hàng đầu:
- Liên kết thông tin: Đối với 10 sản phẩm bán chạy nhất tìm thấy, Backend liên kết (JOIN/Query) với bảng products tại Main DB để lấy thêm thông tin mô tả hiển thị (Tên sản phẩm, đơn giá thực tế).

Hiệu năng & Khả năng chịu lỗi: Trả về kết quả tức thời, hoạt động ổn định bất kể trạng thái kết nối của các Node miền phụ.

5. Xem đơn hàng có nhiều kiện hàng từ các kho khác nhau

5.1. Mô tả yêu cầu nghiệp vụ

Khi khách hàng hoặc quản lý muốn xem chi tiết một đơn hàng, hệ thống cần hiển thị thông tin đầy đủ của đơn hàng đó bao gồm thông tin tổng quan, danh sách các kiện hàng liên quan và chi tiết các sản phẩm trong từng kiện hàng. Do đặc thù hệ thống bán hàng đa kho, một đơn hàng có thể chứa các sản phẩm được phân bổ từ nhiều kho hàng ở các miền khác nhau. Khi đó, đơn hàng được chia tách thành nhiều kiện hàng tương ứng với các kho chịu trách nhiệm xuất hàng. Hệ thống cần truy xuất và tổng hợp tất cả các kiện hàng này từ các cơ sở dữ liệu chi nhánh để hiển thị đầy đủ cho người dùng.

Chi Tiết Đơn Hàng #15

Khách hàng: **Nguyễn Văn Hoàng**
Ngày đặt: **19:10:39 31/5/2026**

Địa chỉ giao: **Số 7k Ngõ 8 Ao Sen**
Tổng tiền: **800.000 đ**

Phân bổ kiện hàng (2 kiện)

Kiện hàng từ: Kho Hà Nội

Pending

Tên Sản Phẩm	Số Lượng	Đơn Giá
Sản phẩm 1	2	100.000 đ
Sản phẩm 2	3	100.000 đ

Kiện hàng từ: Kho TP Hồ Chí Minh

Pending

Tên Sản Phẩm	Số Lượng	Đơn Giá
Sản phẩm 3	3	100.000 đ

5.2. Phân tích dữ liệu phân tán liên quan

- Bảng orders (Đơn hàng): Lưu trữ thông tin tổng quan của đơn hàng (như mã khách hàng, địa chỉ giao hàng, tổng tiền, ngày tạo). Bảng này được lưu trữ tập trung tại cơ sở dữ liệu trung tâm (main_db) để quản lý toàn cục.
- Bảng order_package_shards (Bảng ánh xạ kiện hàng): Đóng vai trò là bản đồ phân mảnh (shard map), lưu trữ thông tin liên kết giữa mã đơn hàng, mã kiện hàng và tên chi nhánh chứa kiện hàng đó (north, central hoặc south). Bảng này được lưu trữ tập trung tại cơ sở dữ liệu trung tâm (main_db).

- Bảng packages (Kiện hàng) và bảng package_details (Chi tiết kiện hàng): Lưu trữ thông tin chi tiết về kiện hàng và các sản phẩm trong kiện. Hai bảng này được phân mảnh ngang dẫn xuất theo kho hàng và lưu trữ cục bộ tại cơ sở dữ liệu của từng chi nhánh tương ứng (north_db, central_db, south_db).
- Bảng warehouses (Kho hàng): Được phân mảnh ngang nguyên thủy và lưu trữ tại 3 database miền (north_db, central_db, south_db). Bảng products (Sản phẩm) và bảng categories (Danh mục sản phẩm): Cung cấp thông tin sản phẩm và danh mục, được lưu trữ tại cơ sở dữ liệu trung tâm (main_db) và nhân bản đầy đủ xuống các chi nhánh.

Thách thức phân tán: Thông tin tổng quan của đơn hàng nằm ở cơ sở dữ liệu trung tâm, nhưng thông tin chi tiết của các kiện hàng lại nằm phân tán ở các cơ sở dữ liệu chi nhánh khác nhau. Để thực hiện truy vấn chi tiết đơn hàng, hệ thống không thể sử dụng câu lệnh nối bảng (JOIN) trực tiếp ở mức cơ sở dữ liệu mà phải sử dụng cơ chế định tuyến truy vấn kết hợp với phép nối ở lớp ứng dụng (application-side join).

5.3. Cơ chế xử lý truy vấn phân tán trong mã nguồn

Hệ thống hiện thực hóa yêu cầu này qua phương thức get_order trong lớp OrderService thuộc Backend như sau:

- Bước 1: Tiếp nhận mã đơn hàng từ yêu cầu của người dùng. Hệ thống thực hiện truy vấn bảng orders trên cơ sở dữ liệu trung tâm để lấy thông tin tổng quan của đơn hàng. Nếu không tìm thấy, hệ thống trả về thông báo lỗi.
- Bước 2: Thực hiện định tuyến truy vấn bằng cách đọc bảng ánh xạ order_package_shards trên cơ sở dữ liệu trung tâm để tìm danh sách các chi nhánh (node_name) chứa các kiện hàng của đơn hàng đó. Điều này giúp hệ thống chỉ kết nối đến các chi nhánh thực sự chứa dữ liệu, tránh việc truy vấn dư thừa sang các chi nhánh khác.
- Bước 3: Đối với mỗi chi nhánh trong danh sách ánh xạ, hệ thống kiểm tra trạng thái hoạt động thông qua bộ ngắt mạch (circuit breaker). Nếu chi nhánh khả dụng, hệ thống mở kết nối đến cơ sở dữ liệu chi nhánh đó để truy vấn thông tin kiện hàng (bảng packages) và chi tiết sản phẩm trong kiện (bảng package_details) thuộc mã đơn hàng tương ứng. Nếu phát hiện chi nhánh ngoại

tuyến hoặc xảy ra lỗi kết nối, hệ thống đánh dấu trạng thái dữ liệu bị khuyết và ghi nhận tên chi nhánh gặp sự cố.

- Bước 4: Thực hiện phép nối tại lớp ứng dụng (application-side join). Hệ thống lấy thông tin bổ trợ về kho hàng (thông qua WarehouseRepository sử dụng cơ chế Local First - Fallback từ các Node phụ), sản phẩm (bảng products) và danh mục sản phẩm (bảng categories) từ cơ sở dữ liệu trung tâm. Sau đó, hệ thống ghép nối các thông tin này với dữ liệu kiện hàng đã thu thập được từ các chi nhánh hoạt động bình thường.

Giải pháp bảo đảm tính sẵn sàng: Nếu một chi nhánh gặp sự cố hoặc ngoại tuyến, bộ ngắt mạch sẽ phát hiện và hệ thống sẽ bỏ qua chi nhánh đó, tiếp tục thu thập dữ liệu từ các chi nhánh hoạt động bình thường nhằm tránh nghẽn luồng. Để bảo đảm tính minh bạch thông tin, hệ thống trả về kết quả kèm cờ is_partial được gán giá trị true và thông điệp warning_message chỉ rõ danh sách các chi nhánh bị mất kết nối, giúp giao diện người dùng hiển thị cảnh báo dữ liệu khuyết một cách chính xác.

6. Người dùng xem danh sách sản phẩm trên hệ thống

6.1. Mô tả yêu cầu nghiệp vụ

Khi khách hàng (hoặc quản trị viên) truy cập vào trang danh sách sản phẩm, hệ thống cần hiển thị toàn bộ danh sách các mặt hàng đang kinh doanh (hỗ trợ lọc theo từng danh mục), cung cấp các thông tin cơ bản của sản phẩm như ID, tên, mức giá và tên danh mục tương ứng

CSDL Phân tán
Xin chào, Đào Xuân Mai

VAI TRÒ & KHU VỰC:
Khách Hàng

Xem Sản Phẩm

Đơn Hàng

Xem Sản Phẩm

Danh mục: -- Tất cả danh mục --

ID	Tên Sản Phẩm	Danh Mục	Giá Bán	Thao Tác
9	Sản phẩm 1	Danh mục 1	100,000	
10	Sản phẩm 2	Danh mục 1	100,000	
11	Sản phẩm 3	Danh mục 2	100,000	
12	Sản phẩm 4	Danh mục 2	100,000	

Hiển thị 1 - 4 trong tổng số 4 bản ghi

< 1 >

- Bảng dữ liệu nhân bản: Bảng products (Sản phẩm) và categories (Danh mục) được áp dụng chiến lược Nhân bản toàn phần từ Main DB xuống cả 3 database miền.
- Thách thức phân tán: Nhờ cơ chế nhân bản toàn phần, dữ liệu sản phẩm giống hệt nhau ở mọi Node, do đó hệ thống không cần gom dữ liệu phức tạp. Thách thức cốt lõi nằm ở việc phải xây dựng cơ chế Cân bằng tải (Load Balancing) để phân bổ lượng lớn yêu cầu đọc danh sách sản phẩm ra các Node chi nhánh nhằm giảm tải cho Main DB, đồng thời phải đảm bảo tính sẵn sàng cao nếu có Node bất ngờ rớt mạng hoặc lỗi.

6.3. Cơ chế xử lý truy vấn trong mã nguồn

Hệ thống kết hợp cơ chế Smart Read Routing (Định tuyến đọc thông minh) để tự động điều phối các luồng yêu cầu và Localized Querying (Truy vấn cục bộ) nhằm khai thác các bảng dữ liệu đã được nhân bản: Hệ thống sử dụng mô hình Load Balancing (Cân bằng tải) thông qua hàm `get_read_db` như sau:

- Bước 1 (Định tuyến Cân bằng tải): Dựa vào Header X-Target-Node do Frontend đính kèm, hàm `get_read_db` của Backend tự động phân luồng. Nếu request đến từ Khách hàng phổ thông, hệ thống sẽ xáo trộn 3 node chi nhánh và chọn ngẫu nhiên một node để chia sẻ tải. Nếu là vai trò Quản trị, quản lý,... hệ thống kết nối thẳng vào Main DB.
- Bước 2 (Kiểm tra - Ping): Trước khi thiết lập kết nối, Circuit Breaker thực hiện một câu lệnh rỗng (`SELECT 1`) để "Ping" kiểm tra xem Node vừa được random có thực sự đang ONLINE hay không.
- Bước 3 (Query): Tại Node an toàn được chọn, Backend thực thi truy vấn lấy thông tin từ bảng products và categories. Quá trình này diễn ra cục bộ, tốc độ truy xuất cực kỳ nhanh và đảm bảo độ nhất quán dữ liệu 100% nhờ đồng bộ nhân bản.
- Khả năng chịu lỗi: Nếu thao tác "Ping" thất bại, Circuit Breaker lập tức ghi nhận Node đó là OFFLINE, hủy kết nối lỗi, phát cảnh báo cho Admin và tự động bẻ lái sang Node chi nhánh tiếp theo. Trong trường hợp xấu nhất, cả 3 chi nhánh đều sập, truy cập sẽ được bẻ lái an toàn về Main DB. Nhờ vậy, trải nghiệm xem danh sách sản phẩm của người dùng luôn xuyên suốt, không bao giờ gặp tình trạng lỗi trắng trang.

PHẦN VI: BÀI TOÁN TRUY XUẤT ĐỒNG THỜI VÀ NHẤT QUÁN TỒN KHO

1. Tra cứu tồn kho của một sản phẩm trên toàn hệ thống

1.1. Phân tích các site truy xuất dữ liệu phân tán

Dữ liệu tồn kho được truy xuất từ các site sau:

- Các site chi nhánh gồm north_db, central_db và south_db: Đây là nơi lưu trữ bảng inventories. Bảng inventories được phân mảnh ngang dẫn xuất dựa trên thuộc tính khu vực địa lý của các nhà kho thuộc bảng warehouses. Mỗi site chi nhánh chịu trách nhiệm lưu trữ và cập nhật số lượng tồn kho thực tế của các nhà kho thuộc khu vực quản lý của mình.
- Site trung tâm main_db: Không lưu trữ dữ liệu tồn kho sản phẩm, do đó không tham gia vào quá trình truy xuất trực tiếp bảng inventories khi thực hiện chức năng này.

1.2. Cách thức tổng hợp dữ liệu

Quy trình tổng hợp dữ liệu tồn kho toàn cục của một sản phẩm được xử lý tại lớp dịch vụ của ứng dụng:

- Bước 1: Truy vấn song song Scatter-Gather: Khi nhận yêu cầu tra cứu thông tin tồn kho cho một mã sản phẩm cụ thể qua product_id, lớp dịch vụ InventoryService sử dụng công cụ ThreadPoolExecutor để khởi tạo các luồng truy vấn song song gửi đến cả ba site chi nhánh.
- Bước 2: Lọc và kiểm tra trạng thái: Trước khi kết nối, hệ thống kiểm tra trạng thái hoạt động của từng site thông qua bộ ngắt mạch Circuit Breaker. Chỉ các site ở trạng thái hoạt động mới được gửi yêu cầu truy vấn.
- Bước 3: Tổng hợp kết quả: Hệ thống gom kết quả trả về từ các luồng chi nhánh, áp dụng cơ chế giới hạn thời gian chờ timeout là 0.5 giây để tránh tắc nghẽn luồng chính nếu có site gặp sự cố. Kết quả sau đó được ánh xạ và gộp thành một danh sách tồn kho đồng nhất hiển thị đầy đủ thông tin về mã nhà kho và số lượng tồn kho khả dụng của sản phẩm trên toàn hệ thống.

1.3. Tác động đến chi phí truyền tải dữ liệu trên mạng

Thiết kế này giúp giảm thiểu đáng kể chi phí truyền tải dữ liệu trên mạng:

- Tối ưu hóa băng thông bằng lọc dữ liệu tại nguồn: Hệ thống chỉ truy vấn các dòng dữ liệu có mã sản phẩm khớp với product_id được yêu cầu tại từng site chi nhánh, tránh việc truyền tải toàn bộ bảng dữ liệu qua mạng.

- Tổng hợp ở lớp ứng dụng: Phép kết hợp dữ liệu được thực hiện ở lớp ứng dụng thay vì thiết lập các mối liên kết cơ sở dữ liệu database link trực tiếp giữa các máy chủ cơ sở dữ liệu, giảm thiểu thời gian chiếm dụng đường truyền.
- Circuit Breaker và kiểm soát thời gian chờ: Loại bỏ các yêu cầu kết nối đến các site đang ngoại tuyến hoặc phản hồi chậm, tránh lãng phí băng thông mạng.

1.4. Cơ chế đảm bảo không âm tồn kho khi có giao dịch đồng thời

Để đảm bảo tính nhất quán và ngăn chặn việc âm tồn kho khi có nhiều đơn hàng đồng thời, hệ thống triển khai cơ chế kiểm soát như sau:

- Tách biệt cơ chế đọc và ghi: Yêu cầu tra cứu tồn kho đơn thuần là một giao dịch chỉ đọc không thực hiện khóa dòng nhằm tối ưu hóa tốc độ phản hồi cho người dùng.
- Khóa bi quan Pessimistic Locking trong giao dịch đặt hàng: Khi có giao dịch đặt hàng diễn ra đồng thời thông qua phương thức `create_order` của lớp `OrderService`, hệ thống sẽ thực hiện truy vấn dòng dữ liệu tồn kho tương ứng của sản phẩm trên các site chi nhánh bằng câu lệnh `SELECT FOR UPDATE` thông qua phương thức `with_for_update` của thư viện `SQLAlchemy`. Việc này khóa chặt các dòng dữ liệu liên quan.
- Xử lý đồng thời tuần tự: Các giao dịch đặt hàng đồng thời khác cố gắng giảm tồn kho của cùng một sản phẩm tại thời điểm đó sẽ bị chặn lại ở mức cơ sở dữ liệu cho đến khi giao dịch trước đó thực hiện `commit` hoặc `rollback`. Điều này đảm bảo số lượng tồn kho được kiểm tra và trừ một cách tuần tự, loại bỏ hoàn toàn rủi ro bán quá số lượng thực tế hoặc gây âm tồn kho.

2. Đặt hàng khi một kho không đủ số lượng và cần kiểm tra khả năng đáp ứng từ kho khác

2.1. Phân tích dữ liệu phân tán và các site truy xuất

Khi khách hàng tạo đơn hàng qua phương thức `create_order` của lớp `OrderService`, hệ thống truy xuất dữ liệu từ các site sau:

Site trung tâm:

- Bảng `users`: Xác thực mã người dùng.

- Bảng products: Kiểm tra thông tin chi tiết sản phẩm và đơn giá.
- Bảng orders và bảng order_package_shards: Ghi nhận thông tin đơn hàng và bản đồ phân mảnh kiện hàng.

Các site chi nhánh:

- Bảng warehouses: Xác định thông tin kho của từng kho hàng.
- Bảng inventories: Kiểm tra và cập nhật tồn kho sản phẩm thực tế tại từng miền.
- Bảng packages và bảng package_details: Chèn dữ liệu kiện hàng và chi tiết sản phẩm tương ứng.

2.2. Cách thức tổng hợp dữ liệu và quy trình phân bổ tồn kho

Quy trình phân bổ tồn kho được xử lý tập trung tại lớp ứng dụng theo cơ chế phân bổ tham lam (Greedy Allocation):

- Bước 1: Hệ thống sử dụng circuit breaker để xác định danh sách các chi nhánh hoạt động (ONLINE), sau đó mở đồng thời các kết nối và giao dịch tới các chi nhánh này.
- Bước 2: Hệ thống truy vấn số lượng tồn kho của sản phẩm từ tất cả các chi nhánh khả dụng, tổng hợp và sắp xếp danh sách nhà kho theo thứ tự số lượng tồn kho giảm dần.
- Bước 3: Thực hiện trừ tồn kho tại nhà kho có số lượng lớn nhất. Nếu không đủ, hệ thống lấy toàn bộ tồn kho tại kho đó và tiếp tục trừ sang nhà kho có tồn kho lớn kế tiếp cho đến khi đáp ứng đủ số lượng yêu cầu.
- Bước 4: Tạo đơn hàng tại site trung tâm, chèn thông tin các kiện hàng tương ứng tại các chi nhánh được phân bổ, đồng thời lưu trữ thông tin định tuyến kiện hàng tại site trung tâm.

2.3. Tác động đến chi phí truyền tải dữ liệu trên mạng

Thiết kế phân tán này giúp tối ưu hóa chi phí truyền tải dữ liệu trên mạng:

- Giảm tải băng thông: Hệ thống thực hiện lọc dữ liệu tồn kho theo mã sản phẩm cụ thể tại từng chi nhánh trước khi gửi về lớp ứng dụng, tránh truyền tải toàn bộ bảng inventories.

- Xử lý liên kết tại lớp ứng dụng: Việc kết hợp thông tin sản phẩm và phân bổ kho hàng được thực hiện ở lớp service thay vì cấu hình liên kết cơ sở dữ liệu trực tiếp giữa các database server, hạn chế chiếm dụng băng thông mạng kéo dài.
- Cơ chế circuit breaker: Giúp phát hiện sớm các chi nhánh ngoại tuyến để bỏ qua việc gửi yêu cầu kết nối, ngăn chặn nghẽn mạng và lãng phí tài nguyên đường truyền.
- Giao dịch định hướng: Chỉ các lệnh cập nhật tồn kho (UPDATE) và tạo kiện hàng (INSERT) thực sự phát sinh mới được gửi qua mạng đến các chi nhánh liên quan.

2.4. Cơ chế đảm bảo không âm tồn kho khi có giao dịch đồng thời

Hệ thống kết hợp hai cơ chế kiểm soát đồng thời để đảm bảo tính nhất quán dữ liệu tồn kho:

- Khóa bi quan (Pessimistic Locking): Khi truy vấn bảng inventories để kiểm tra số lượng, hệ thống sử dụng câu lệnh SELECT FOR UPDATE thông qua phương thức with_for_update() của SQLAlchemy để thực hiện khóa dòng dữ liệu. Giao dịch đồng thời khác muốn đọc hoặc ghi trên cùng dòng sản phẩm tại chi nhánh đó sẽ phải xếp hàng chờ đợi, đảm bảo dữ liệu đọc ra luôn là số liệu mới nhất.
- Giao dịch phân tán mức ứng dụng (Simulated Two-Phase Commit): Phiên làm việc trên site trung tâm và các site chi nhánh được quản lý đồng bộ. Nếu tổng số lượng tồn kho toàn hệ thống không đủ đáp ứng, hoặc xảy ra lỗi kết nối tại bất kỳ chi nhánh nào, hệ thống lập tức gọi lệnh rollback() đồng loạt trên tất cả các site để hoàn tác và giải phóng các dòng tồn kho bị khóa. Lệnh commit() chỉ được thực thi đồng thời khi toàn bộ quy trình tại mọi site thành công, loại bỏ hoàn toàn nguy cơ âm tồn kho hoặc bán quá số lượng thực tế.

PHẦN VII: ĐÁNH GIÁ HỆ THỐNG

1. Đánh giá hiệu năng hệ thống

1.1 Độ trễ (Latency)

a. Đối với tác vụ đọc (Read Latency)

- Thông tin sản phẩm và danh mục được sao chép sẵn tại các máy chủ miền, giúp các truy vấn xem sản phẩm và danh mục được thực hiện cục bộ mà không cần truy vấn đến máy chủ trung tâm.
- Các báo cáo doanh thu được tổng hợp và lưu trữ trước tại máy chủ trung tâm, do đó giao diện thống kê có thể hiển thị kết quả gần như tức thời

b. Đối với tác vụ ghi (Write Latency)

- Khi tạo đơn hàng, hệ thống cần đồng bộ với các máy chủ miền để kiểm tra và cập nhật tồn kho.
- Do phải trao đổi dữ liệu qua mạng giữa nhiều nút, thời gian xử lý giao dịch sẽ cao hơn so với hệ thống cơ sở dữ liệu tập trung.

1.2 Thông lượng (Throughput)

- Dữ liệu tồn kho được phân mảnh theo từng khu vực Bắc, Trung và Nam.
- Các giao dịch phát sinh tại từng miền được xử lý trên máy chủ tương ứng, hạn chế tình trạng tranh chấp tài nguyên trên một cơ sở dữ liệu duy nhất.
- Nhờ đó, hệ thống có thể xử lý số lượng lớn đơn hàng đồng thời và đạt thông lượng cao hơn so với mô hình tập trung.

1.3 Khả năng mở rộng (Scalability)

Khả năng mở rộng theo chiều ngang

- Khi phát sinh thêm khu vực hoạt động (ví dụ: Miền Tây), chỉ cần:
 - + Khởi tạo một DB Node mới.
 - + Khai báo thông tin trong file cấu hình (.env).
- Không cần thay đổi cấu trúc dữ liệu hay mã nguồn của các node hiện có.
- Việc phân tán tải ghi và quản lý tồn kho tại các node địa phương giúp giảm áp lực lên Main DB, hạn chế tình trạng quá tải khi số lượng đơn hàng tăng mạnh.

1.4 Tính sẵn sàng (Availability)

- Khi một máy chủ miền gặp sự cố, các khu vực còn lại vẫn hoạt động bình thường.

- Các yêu cầu liên quan đến miền gặp lỗi sẽ được chuyển hướng về máy chủ trung tâm hoặc được xử lý riêng biệt, tránh ảnh hưởng đến toàn bộ hệ thống.
- Cơ chế này giúp tăng khả năng chịu lỗi và giảm nguy cơ xảy ra điểm lỗi đơn lẻ (Single Point of Failure).

1.5 Mức độ sử dụng tài nguyên (Resource Utilization)

Tối ưu tài nguyên xử lý

- Các phép tính thống kê được thực hiện tại từng máy chủ miền trước khi gửi kết quả tổng hợp về máy chủ trung tâm.
- Cách tiếp cận này giúp phân tán tải xử lý, giảm áp lực CPU và RAM cho máy chủ trung tâm.

Tối ưu băng thông mạng

- Dữ liệu sản phẩm được lưu trữ cục bộ tại các miền, hạn chế việc truyền dữ liệu lặp lại qua mạng.
- Chỉ các kết quả tổng hợp cần thiết mới được đồng bộ về trung tâm, giúp tiết kiệm băng thông.

Hạn chế

- Việc vận hành đồng thời nhiều máy chủ cơ sở dữ liệu làm tăng nhu cầu sử dụng tài nguyên phần cứng, đặc biệt là bộ nhớ RAM và dung lượng lưu trữ, so với mô hình cơ sở dữ liệu tập trung.

2. So sánh phương án tập trung với phân tán

Để đánh giá hiệu năng thực tế, nhóm nghiên cứu đã thử nghiệm tải cao Load Testing bằng Grafana k6, giả lập truy cập đồng thời lớn và sự cố sập nút.

2.1. Thiết lập thử nghiệm

- Môi trường: Dựng bằng Docker Compose gồm backend csdlpt_be (FastAPI, pool_size=30, max_overflow=20) kết nối tới cơ sở dữ liệu Tập trung (csdlpt_db) hoặc cơ sở dữ liệu Phân tán (3 node chi nhánh: mysql_node1, mysql_node2, mysql_node3).
- Kịch bản k6: API GET /product, tăng dần từ 0 lên 150 VUs trong 15 giây, nghỉ 0.1 giây giữa mỗi request.

- Định tuyến đọc: Tập trung gửi về Main DB; Phân tán chia tải ngẫu nhiên qua các Regional DBs có Circuit Breaker giám sát.
- Giả lập sự cố: Tắt đột ngột một máy miền (mysql_node1) bằng lệnh docker compose kill mysql_node1 trước khi thực hiện kiểm thử để đánh giá phản ứng ngay từ đầu của hệ thống. Cấu hình kết nối connect_timeout đặt ở mức 0.5 giây.
- Khử nhiễu: Khởi động lại container backend trước mỗi lượt test để làm sạch bộ nhớ và hàng đợi kết nối.

2.2. Kết quả đo lường thực tế

Bảng Kết Quả So Sánh Hiệu Năng			
Chỉ số	Tập trung (Centralized)	Phân tán (Distributed)	Phân tán khi sập 1 Node (Failover)
Tổng số Request	2862	2922	2576
Throughput (Tốc độ xử lý)	143.06 req/s	145.40 req/s	128.40 req/s
Độ trễ trung bình (Avg Latency)	384.03 ms	372.24 ms	437.45 ms
Độ trễ phân vị P90	824.87 ms	823.97 ms	756.00 ms
Độ trễ phân vị P95	891.47 ms	922.09 ms	981.10 ms
Tỷ lệ lỗi (Error %)	0.00%	0.00%	1.82%
Trạng thái hoạt động	Bình thường	Bình thường	Tự phục hồi (Auto-recovered)

2.3. Phân tích kết quả thực nghiệm

a. So sánh Phân tán và Tập trung

- Thông lượng: Phân tán đạt 145.40 req/s, cao hơn Tập trung (143.06 req/s). Do cơ chế nhân bản bảng sản phẩm xuống các chi nhánh giúp chia đều tải đọc, khai thác tốt năng lực CPU và I/O của 3 máy chủ độc lập thay vì nghẽn tại Main DB.
- Độ trễ: Phân tán đạt độ trễ trung bình tối ưu hơn (372.24 ms so với 384.03 ms của Tập trung). Việc tắt pool_pre_ping ở các Regional DBs giúp giảm tải truy vấn ping kiểm tra kết nối, tối ưu hóa thời gian phản hồi.

b. Khả năng tự phục hồi

Khi mysql_node1 bị sập đột ngột:

- Chỉ có 1.82% request (47 request) gặp lỗi tại thời điểm sập do mất kết nối vật lý tức thời.
- Nhờ connect_timeout tối ưu chỉ 0.5 giây, Backend nhanh chóng phát hiện lỗi, cô lập node sập và chuyển trạng thái sang OFFLINE trên Circuit Breaker.
- Toàn bộ request tiếp theo được bẻ lái tự động sang các node còn sống mà không cần khởi động lại server. Tốc độ trung bình cả bài test Failover vẫn đạt mức cao (128.40 req/s).
- Nếu sập Main DB ở mô hình tập trung, tỷ lệ lỗi sẽ là 100%.

2.4. Kết luận

Thử nghiệm chứng minh mô hình phân tán cho hiệu năng đọc cao hơn và độ trễ thấp hơn khi chịu tải lớn. Đồng thời, cơ chế Circuit Breaker với connect_timeout 0.5 giây giúp hệ thống tự cô lập sự cố và phục hồi tức thời.

3. Đề xuất cải tiến hệ thống

3.1. Xử lý trừ kho song song

Vấn đề: Hiện tại máy chủ Trung tâm đang kết nối lần lượt từng miền một để kiểm tra và trừ tồn kho. Nếu một máy miền phản hồi chậm, khách hàng sẽ phải đợi lâu để biết kết quả đặt hàng.

Đề xuất: Cải tiến để hệ thống gửi yêu cầu kiểm tra và trừ kho đồng thời (song song) tới tất cả các máy miền cùng một lúc, giúp rút ngắn tối đa thời gian chờ đợi của khách hàng khi nhấn nút đặt mua.

3.2. Hộp thư nháp tự động

Vấn đề: Khi xảy ra sự cố mất kết nối mạng ở một máy miền, các thông tin đồng bộ từ máy chủ Trung tâm bị lỗi và phải đợi hệ thống chính chạy tiến trình quét lại khá lâu để gửi lại.

Đề xuất: Thiết lập một "hộp thư nháp" tạm thời để cất dữ liệu đồng bộ khi máy miền mất kết nối. Ngay sau khi phát hiện máy miền đó hoạt động trở lại, toàn bộ dữ liệu đang chờ trong hộp thư nháp sẽ được tự động gửi đi ngay lập tức.

3.3. Sử dụng bộ nhớ đệm (Caching) cho danh sách sản phẩm

Vấn đề: API lấy danh sách sản phẩm GET /product có tần suất truy vấn rất lớn. Việc truy vấn trực tiếp vào cơ sở dữ liệu SQL dưới tải trọng lớn tiêu tốn nhiều tài nguyên I/O và CPU của các database node.

Đề xuất: Tích hợp bộ nhớ đệm Redis tại mỗi Backend Node để lưu trữ danh sách sản phẩm. Khi khách hàng gọi API GET /product, hệ thống sẽ ưu tiên đọc dữ liệu từ Redis Cache. Dữ liệu cache sẽ được tự động xóa khi Admin thực hiện thay đổi thông tin sản phẩm ở database Trung tâm, đảm bảo tính nhất quán dữ liệu và tối ưu hóa hiệu năng hệ thống.